



TECH2

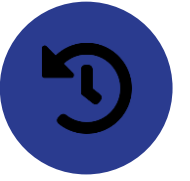
Week 2 Workshop

Version Control with Git



A Familiar Problem

- You're working on a script. It's working. You keep going.
- You make a “quick change” — and now it's broken.
- You can't quite remember what you changed, or what the working version looked like.
- Sound familiar? This happens to every programmer — including professionals, every day.
- Git exists to solve exactly this problem!



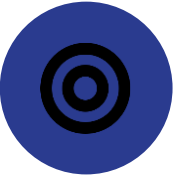
What Is Git?

- Git saves numbered snapshots of your folder, over time.
- Each snapshot is called a **commit**.
- You decide when to take one — nothing is saved automatically.
- Because every snapshot is kept, you can always go back to an earlier one.



Why This Matters

- Git is the industry standard for tracking code — used by virtually every professional software and data team.
- Today, you'll only use the basics: saving snapshots on your own computer.
- Even the small habit you build today — commit, commit, commit — is the same habit used by professional developers every single day.
- Git also connects to remote services like GitHub, where a repository can be stored online and shared with others.
- Later in this course, you'll use GitHub directly — including forking a repository for the second half of the course.



Today's Plan

- Understand what git actually does, and why it's worth learning
- Set up a git repository for this week's folder, right inside VS Code
- Save your first snapshots (commits) as you solve exercises
- Break something on purpose — and recover it using your commit history

Goal for today: commit after every exercise becomes second nature — something you do without thinking, from week 3 onward.



Set Up This Week's Folder

1. In VS Code, open your week02 folder (File → Open Folder), if it isn't open already
 2. Open the Source Control panel (icon in the left sidebar, or Ctrl+Shift+G / Cmd+Shift+G)
 3. Click Initialize Repository
 4. Git is now watching this folder. It does this by creating a hidden .git folder inside week02 – you won't see it by default, since it's hidden
- How to see hidden folders:

Windows

File Explorer → View tab → check “Hidden items”

Mac

Finder → Cmd+Shift+. (period)

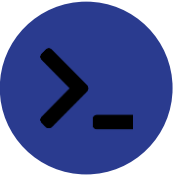


One More Setup Step: .gitignore

- In week02, create a new file named exactly .gitignore
- This tells git to permanently ignore certain files — never track them, never show them as changes
- Add the following items to your .gitignore file (or copy-paste from Github):

Pattern	Why we ignore it
*.pptx / *.pdf	Slides and handouts — reference material, not your work
.vscode/	VS Code's own project settings, specific to your computer
__pycache__ / *.pyc	Python's compiled bytecode — regenerated automatically
.ipynb_checkpoints/	Jupyter's autosave backups — clutter, not real changes
.DS_Store / Thumbs.db	Hidden files your OS creates to remember folder view settings

Note: Like `git init`, this is a one-time setup step — do it once for each git repo.



Tell Git Who You Are

- 1 Git records who made each commit — so it needs your name and email before you can commit anything
- 2 In Git Bash (Windows) or Terminal (Mac), run: `git config --global user.name "Your Name"`
- 3 Then run: `git config --global user.email "your.email@example.com"`
- 4 Check it worked: `git config --global --list`

Use the email you'll use for GitHub — in part 2 it links your commits to your account. And --global means once per computer, not once per week.



The Three States



- **Changes** — files you've edited since the last commit, listed under "Changes" in Source Control
- **Staged Changes** — files marked as ready to save, by clicking the + next to a file
- **Commit** — the actual snapshot: write a message, click ✓ Commit



Exercise 1 — Divide Two Integers

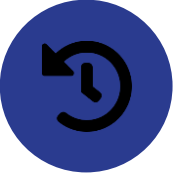
- 1 In week02, create a new file: ex1.py
- 2 Write a program that prompts for two integers (input), divides the first by the second, and displays the result with one decimal
- 3 Run it (▶ Play button) and check the output
- 4 Open Source Control — see ex1.py listed under Changes
- 5 Click the + next to ex1.py to stage it
- 6 Type a commit message (e.g. “ex1: attempt”), then click ✓ Commit

Tip: Before doing anything else in a project, glance at the Source Control panel — it always tells you exactly what's changed.



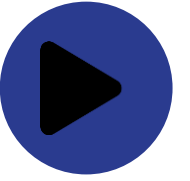
Exercise 2 — Divide Two Floats

- 1 Create ex2.py
- 2 Same task as Exercise 1, but with float values, displayed with two decimals
- 3 Run it, confirm the output
- 4 Stage it and commit it — same steps as before



Looking Back: Your Timeline

- Every file has a Timeline — right-click a file → Open Timeline (or the Timeline view in the Explorer sidebar)
- It lists every commit that touched this file, most recent first
- Click any entry to see exactly what changed — a diff, red for removed, green for added
- You can also restore a file to how it looked at that point
- This is what makes git useful: not just saving snapshots, but being able to actually look at and return to them



Exercise 3 — Random Number Generator

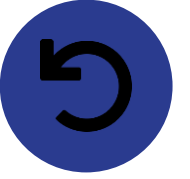
- 1 Create ex3.py
- 2 Write a program that prompts for a lower and upper bound, uses randint from the random module to draw a random integer between them, and displays it
- 3 Run it, confirm it works
- 4 Stage and commit it — message: “ex3: working version”

Note: Get this version safely committed — we're about to do something risky.



Now Let's Break It

- 1 In ex3.py, delete the code that converts upper to an integer (the `int(...)` around the input)
- 2 Run the program again — it crashes: `randint` received text, not a number
- 3 Don't fix it from memory. Don't guess. Don't undo blindly. This is exactly the situation git was built for!

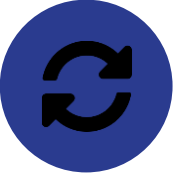


Recovering the Working Version

- 1 Open ex3.py's Timeline (right-click on file in Explorer -> Open timeline)
- 2 Find the commit “ex3: working version” — click it
- 3 Review the diff — confirm this is the version before the bug
- 4 Fix the file
- 5 Run ex3.py again — it works

Tip: To avoid manually fixing files, you can use *git* in the terminal. Copy the commit hash (right-click the commit → Copy commit hash) and run the following command in the terminal:

```
git restore --source=<commit-hash> -- filename
```



The Loop

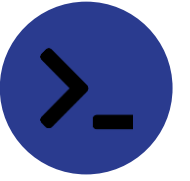
- Edit → Stage (+) → Commit (✓)
- Commit after every exercise — whether it's finished or not
- “Am I happy with it yet?” is the wrong question. “Did I just finish something?” is the right one.
- Do this every workshop, every week, for the rest of the course



Exercise 4 — Fahrenheit to Celsius

- 1 Create ex4.py
- 2 Write a program that prompts for a Fahrenheit temperature, converts it using $\text{Celc} = (5/9) \times (\text{Fahr} - 32)$, and displays the result rounded to the nearest integer
- 3 Run it, confirm the output
- 4 Stage and commit it

***Tip:** Instead of downloading a solution proposal, type it up yourself from scratch. Tedious, yes — but it's how the code actually sticks.*



Same Thing, Typed

VS Code action	Terminal command
Initialize Repository	<code>git init</code>
File under Changes → click +	<code>git add <file></code>
✓ Commit (with a message)	<code>git commit -m "message"</code>
Timeline view	<code>git log</code>

Tip: If you ever want to start a repo over from scratch, delete the hidden `.git` folder inside your week's folder — this removes all commit history but leaves your actual files untouched. To start tracking again, just click Initialize Repository as before



What You've Built Today

- week02 — a git repository, initialized and gitignored
- ex1.py, ex2.py, ex3.py, ex4.py — four scripts, each committed
- A recovered version of ex3.py, restored from history
- The one habit to keep: commit after every exercise, starting next week

**Next week: back to Python —
but from now on, always with git running quietly in the background.**